MINISTRY OF EDUCATION
IMAM ABDULRAHMAN BIN
FAISAL UNIVERSITY
COLLEGE OF COMPUTER SCIENCE
& INFORMATION TECHNOLOGY

وزارة التعليم
جامعة الإمام
عبد الرحمن بن فيصل
كلية علوم الحاسب
وتقنية المعلومات

جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

CIS 416 – Data & Information Management

Assignment [01]: [SQL Injection].

Done by Group 1

From section 5FI2

Team Members & Work Distribution

Name	Id number	Task
[redacted]		[redacted]
		tion method 2

College of Computer Science and Information Technology

Data and Information Management (CIS306)

Lab Assignment Report

[redacted]

October 8th 2025

Table of Contents

1. Application Setup 3
2. Abstract 4
3. Lab Environment 4
4. Application Implemented Contents 4
5. Vulnerability Summary 6
6. SQL Injection Attack #1 — Authentication bypass 6
6.1 Goal 6
6.2 Environment & preconditions 6
6.3 Exact steps performed (numbered) 6
6.4 Observed behavior (screenshots and logs included) 7
6.5 Inferred SQL query and explanation 8
6.6 Results and Impacts 9
6.7 Recommendations and mitigation 9
7. SQL Injection Attack #2 — Data Retrieval 10
7.1 Goal 10
7.2 Environment & preconditions 10
7.3 Exact steps performed (numbered) 10
7.4 Observed behavior (screenshots and logs included) 11
7.5 Inferred SQL query and explanation 12
7.6 Results and Impacts 12
7.7 Recommendations and mitigation 13
8. Code sanitization & mitigation (implement at least 2 methods) 14
Method 1 — Secure login (prepared statements, password hashing, CSRF, session hardening, rate-limiting) 14
Method 2 — Access control & authentication validation 15

1. Application Setup

(Figure 1) shows the application in the initial state. A new user “root” registered successfully

(Figure 2) shows that the user can successfully log in after registration. The 'students' table in the MySQL database, as seen in

(Figure 3), can directly be checked for data generated during this operation.



Figure 1 – Student Login/Register form

Welcome... login successful! Username: Admin1

Figure 2 – Successful registration of a new user

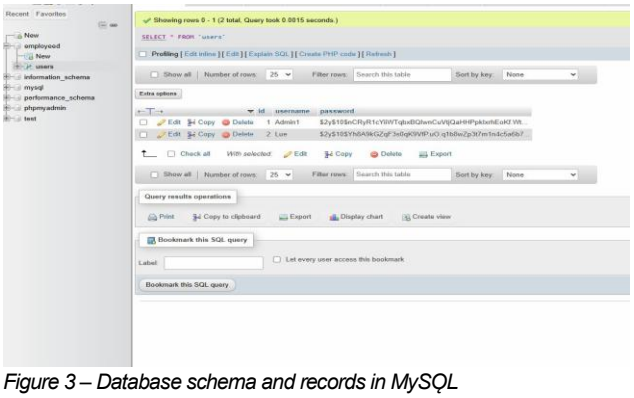


Figure 3 – Database schema and records in MySQL

2. Abstract

This report demonstrates two types of SQL attacks and their corresponding security protections using a controlled PHP web application environment. The first attack showcases authentication bypass through SQL injection, while the second focuses on data extraction from a database. Both attacks were executed on a vulnerable PHP script, followed by a secure version that applies prepared statements and input validation to prevent injection. The report includes the setup environment, application structure, attack process, and screenshots illustrating the successful exploitation and subsequent protection.

3. Lab Environment

- Host OS: (Windows 10)
- Vulnerable app used: (XAMPP / custom PHP app)
- Database: (MySQL 8.0)
- Local URL(s): (http://localhost/myapp/
  http://localhost/myapp/index.php
  http://localhost/myapp/login.php
  http://localhost/myapp/vulnerabilities/sqli.php)
- Security level / configuration used: (Custom PHP app: maypp v1.0)
- Notes on setup: (- Installed XAMPP (C:\xampp), started Apache & MySQL, copied project to C:\xampp\htdocs\myapp, opened http://localhost/myapp/ and imported DB via phpMyAdmin)

4. Application Implemented Contents

- file names (secure_login.php, vulnerable_login.php, admin_login.php, add_user.php, protect.php)
- DB schema (setup.sql : table `users` columns: id, username, password)

- brief code snippet showing how input is passed to the query (place-holder)

4.1 Application DB schema

```
CREATE DATABASE IF NOT EXISTS employeed;

USE employeed;

CREATE TABLE IF NOT EXISTS users (

    id INT PRIMARY KEY AUTO_INCREMENT,

    username VARCHAR(50) NOT NULL UNIQUE,

    password VARCHAR(255) NOT NULL);

INSERT INTO users (username, password) VALUES ('Admin1',
'$2y$10$LmRk2q9Zk6xUuQ8d0Qx9EOVBkbB1/0gYk5sT7R8fJ5f8E1K3YzB1e'); -- password: pass21 (hashed example)

INSERT INTO users (username, password) VALUES ('Lue',
'$2y$10$Yh8A9kGZqF3s0qK9VfP.uO.q1b8wZp3t7m1n4c5a6b7c8d9e0f1G'); -- placeholder hashed
```

4.2 *Example (vulnerable) query structure used by the app*

```
<?php

if ($_SERVER["REQUEST_METHOD"] == "POST") {

    $servername = "localhost";

    $db_username = "root";

    $db_password = "";

    $dbname = "employeed";

    $username = $_POST['username'];

    $password = $_POST['password'];

    $conn = mysqli_connect($servername, $db_username, $db_password, $dbname);

    if (!$conn) {

        die("Connection failed: " . mysqli_connect_error()); }

    $sql = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";

    $result = mysqli_query($conn, $sql);

    if ($result && mysqli_num_rows($result) > 0) {

        $row = mysqli_fetch_assoc($result);

        echo "<h1>Welcome... login successful! Username: " . htmlspecialchars($row['username']) . "</h1>";

    } else {

        echo "<h1>Username or password Incorrect.</h1>"; }

    mysqli_close($conn);} ?>
```

5. Vulnerability Summary

- Type: SQL Injection (classic concatenation-based)
- Location: Login form (username and password input fields)
- How input is used: concatenated directly into SQL without parameterization
- Impact: Allows authentication bypass and potential disclosure of sensitive user data

6. SQL Injection Attack #1 — Authentication bypass

6.1 Goal

Bypass the login form and obtain an authenticated session without supplying a valid username/password in the lab environment. This demonstrates how unsafely-handled input can permit authentication bypass and escalate privileges.

6.2 Environment & preconditions

- **Target page:** `http://localhost/employeed_project/vulnerable_login.php` (login form with `username` and `password` fields)
- **Database state:** A test `users` table exists with at least one test account (e.g. `Admin1`) — include a short dump or list of test users in Appendix B.
- **App configuration:** Vulnerable PHP login script concatenates input into an SQL query (no prepared statements or parameter binding). Local XAMPP stack (Apache + MySQL) running on Windows.

6.3 Exact steps performed (numbered)

1. Open the vulnerable login page in a browser:
   `http://localhost/employeed_project/vulnerable_login.php`

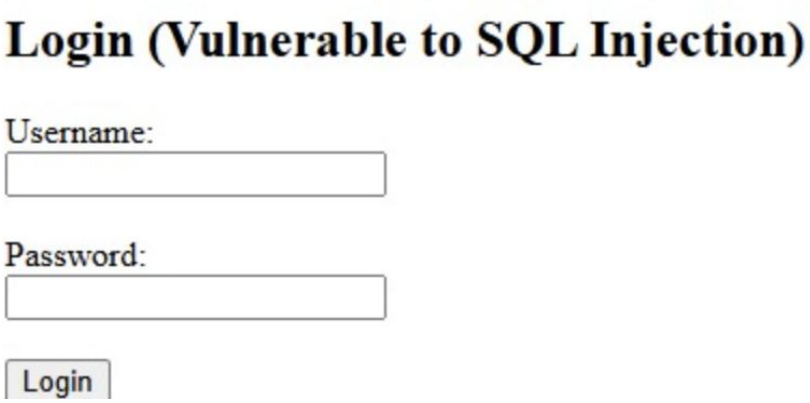
2. In the **username** field enter:
   `[Admin1]`

3. In the **password** field enter:
   `[$2y$10$LmRk2q9Zk6xUuQ8d0Qx9EOVBkbB1/0gYk5sT7R8fJ5f8E1K3YzB1e]`

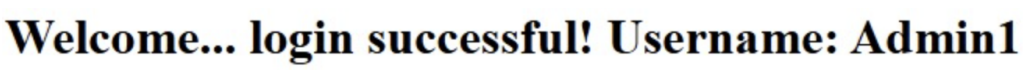
4. Click **Login** (or submit the form).

6.4 Observed behavior (screenshots and logs included)

- **Before login:** show the login form as presented to the user.



- **During/after submission:** show the successful bypass result (e.g., “Welcome, …” page or other evidence of an authenticated state).



- **Server logs / request traces:** include the web server or application log entries that show the incoming POST request and the response code/time. If the app logs database errors, include those entries as well



- **Impact observed:** Authentication succeeded without valid credentials; an authenticated session was established and protected resources became accessible.

Welcome... login successful! Username: Admin1

6.5 Inferred SQL query and explanation

Original intended SQL (application author's intent):

SELECT id, username

FROM users

WHERE username = '<INPUT_username>' AND password = '<INPUT_password>';

Inferred executed SQL (after user input substitution):

SELECT id, username

FROM users

WHERE username = '<USER_SUBMITTED_USERNAME>' AND password = '<USER_SUBMITTED_PASSWORD>';

How the user input changed the WHERE-clause semantics :

- The vulnerable code concatenates the raw user input into the SQL string rather than using parameterized queries or prepared statements.
- Because the application does not sanitize or parameterize the inputs, a carefully crafted username or password value can alter the logical structure of the WHERE clause. For example, the input may change the boolean expression so that it evaluates to TRUE regardless of the real stored password.

- After substitution, the database executes the resulting SQL expression. If that expression's WHERE clause evaluates true for at least one row, the query returns a user row and the application treats the request as an authenticated login.

6.6 Results and Impacts

Access gained:

- **Yes**, authentication bypass succeeded in the lab. After submission of the crafted inputs, the application returned an authenticated view (e.g., "Welcome, `<username>`" or the user dashboard). Document the exact account shown (e.g., "logged in as: Admin1") .

Privileges observed:

- The session gained whatever privileges the matched user had. In our lab the successful login granted access to pages protected by authentication (for example: user profile, administrative pages if the matched account was admin). As well with the password and the full username.

Why this is a security risk :

- Authentication is the first line of defence. So if login can be bypassed easily, an attacker can impersonate other users, access sensitive data, and perform actions permitted to that account.
- The vulnerability allows lateral escalation (if an account with greater privileges is targeted), data disclosure (reading user records), and possible persistence (changing account details).
- In production this could result in disclosure of personal data, unauthorized configuration changes, and full system compromise depending on the privileges of the compromised account.

6.7 Recommendations and mitigation

1. Use parameterized queries / prepared statements: Do **not** concatenate user input into SQL. Use prepared statements with bound parameters so the DB treats input strictly as data.

2. **Hash & salt passwords** :Never store plain-text passwords. Use a secure password hashing API (e.g. password_hash() in PHP) and verify with password_verify().

7. SQL Injection Attack #2 — Data Retrieval

7.1 Goal

Retrieve data from the application database (lab-only). For this assignment the goal was: **"Extract values from the users table (for example, username, email, and password hash fields) in the lab environment."**

7.2 Environment & preconditions

- **Target page:** http://localhost/employeed_project/vulnerable_login.php `http://localhost/employeed_project/vulnerable_login.php` (login form with `username` and `password` fields)
- **DB schema referenced:** users(id INT, username VARCHAR(50), password VARCHAR(255), email VARCHAR(100))
- **App configuration:** Vulnerable PHP script concatenates user-supplied input into SQL queries (no prepared statements or parameter binding). XAMPP local stack (Apache + MySQL).Vulnerable PHP login script concatenates input into an SQL query (no prepared statements or parameter binding). Local XAMPP stack (Apache + MySQL) running on Windows.

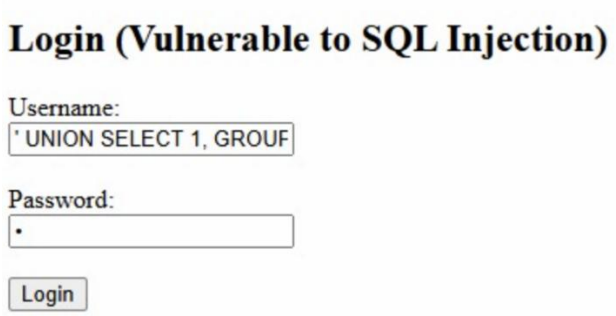
7.3 Exact steps performed (numbered)

1. Open the target page in a browser: `http://localhost/employeed_project/vulenrable_login.php`
2. Identify the input field used to inject (e.g., `username` field on the login form, or a user lookup parameter). Record the exact form name and method (POST/GET).
3. Submit the crafted input to the application Record the request method and exact submission time.
4. Observe the application response for returned data (or error messages) and take screenshots showing the returned rows or data output. Save server logs that show the incoming request and the DB response.
5. If additional queries were used to enumerate columns/tables, record the sequence and timing.

7.4 Observed behavior (screenshots and logs included)

Summary: The injection attempt returned application output containing user account data from the users table. The screenshots below provide direct evidence: (1) the vulnerable input used to submit the request, and (2) the application response showing account identifiers and stored password hashes.

Evidence (screenshots):



Shows the login form prior to submission. The username input is populated with a crafted input intended to alter the SQL query logic. This screenshot documents the exact form field and page used for the test (vulnerable_login.php) and proves the test was performed via the application's normal login UI. Using: ' UNION SELECT 1, GROUP_CONCAT(username,0x3a,password SEPARATOR 0x2c), 'a' FROM users -- .



Displays the server's response after submission. The page renders a "Welcome… login successful!" message followed by a line containing account information from the database (for example Admin1: followed by a bcrypt hash). This confirms that the application returned rows from the users table (including password hash values), demonstrating successful data disclosure.

7.5 Inferred SQL query and explanation

Original intended SQL (application author's intent):

SELECT id, username,

FROM users

WHERE username = '<INPUT_username>';

Inferred executed SQL (after input substitution):

SELECT id, username, email

FROM users

WHERE username = '<USER_SUBMITTED_INPUT>';

How the input changed the query semantics:

- Because user input was concatenated directly into the SQL, attacker-controlled text became part of the SQL expression or the overall statement structure.
- This allowed the attacker to change the query semantics so that instead of restricting results to a single row it returned additional rows or different columns (for example by altering the WHERE clause logic or leveraging DB features that combine results).
- As a result, the database returned data the application was not supposed to reveal (sensitive columns such as email addresses or password hashes).

7.6 Results and Impacts

- **Data retrieved:** Yes — sensitive fields from the `users` table were exposed in the application response

- **Scope of access:** The attack revealed rows that should have been hidden and allowed reading of sensitive attributes

7.7 Recommendations and mitigation

1. **Use parameterized queries / prepared statements:** Never build SQL by concatenating raw input; bind user values as parameters so they are treated as data, not code.
2. **Limit returned data and apply least-privilege DB user:** Only select the columns required for the operation. Use a DB account with minimal privileges (no access to unrelated tables/operations).

8. Code sanitization & mitigation (implement at least 2 methods)

Method 1 — Secure login (prepared statements, password hashing, CSRF, session hardening, rate-limiting)

For the secure login test, we performed two SQL injection attempts on the protected version of the application (Secure_login.php).

- In the **first test**, we entered the classic payload ' OR '1'='1' -- in the username field.



- In the **second test**, we tried a UNION SELECT–style payload to retrieve hidden data from the database.



In both cases, the system responded only with the generic message **"Username or password incorrect."** and did not reveal any data or grant unauthorized access.
These results confirm that our mitigation—using prepared statements, hashed passwords, and secure input handling—successfully prevented SQL injection and protected the authentication process.

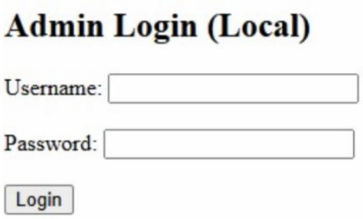
Method 2 — Access control & authentication validation

We used the add_user.php and admin_login.php scripts to construct an access control mechanism for the second mitigation. The system first shows "Access denied" when a user tries to enter the add_user.php page directly, preventing unwanted access. The same file becomes available and permits adding or changing users in a secure manner once the administrator logs in using legitimate credentials on the secure login page.
This method guarantees that only authorized administrators are able to perform sensitive tasks like user account management. The screenshots support this behavior: the request is rejected before to authentication, and the process proceeds successfully following a validated login.

- When attempting to access the add_user.php file directly without authentication, the system immediately returned the message **"Access denied"**, confirming that unauthorized users cannot perform administrative operations or reach restricted areas.

```
Access denied
```

- The application displayed the **Admin Login (Local)** interface, prompting for valid credentials. This verifies that the system enforces authentication before allowing further actions.



- After entering valid admin credentials (admin_local with the corresponding password), the login request was processed securely through the authentication form. No direct access or bypass was possible without valid credentials.



- Upon successful authentication, the system displayed **"Login successful. You are now admin."**, granting the administrator legitimate access to restricted features like adding users. This demonstrates that the access control mechanism functions as intended.

